



PYTHON NOTES — PYTHON DICTIONARIES AND TUPLES



TOPIC: DICTIONARIES

Python dictionaries are one of the most powerful and useful data structures in Python. They allow us to store information using **keys** and **values**, making it easy to organize and retrieve related pieces of data.

Unlike lists and tuples, dictionaries do **not** use positions (indexes) to access information. Instead, they use **keys**.

Dictionaries are used in many real-world Python programs, such as:

- Student Management Systems
- Library Management Systems
- Contact Books
- Inventory Systems
- Banking Applications
- User Accounts
- APIs
- Databases
- Configuration Files

Because of their flexibility, dictionaries are among the most commonly used data structures in Python.

1. WHAT IS A DICTIONARY?

A dictionary is a collection of information stored as **key-value pairs**.

Every piece of information has:

- a **key**
- a **value**

Think of a real dictionary:

Word → Meaning

Python dictionaries work in a similar way:

Key → Value

Example:

```
student = {  
    'name': 'Christabel',  
    'age': 14,  
    'school': 'Silver Fountain College'  
}
```

Here:

```
'name'    → 'Christabel'  
'age'     → 14  
'school'  → 'Silver Fountain College'
```

Each key points to its corresponding value.

2. CREATING A DICTIONARY

A dictionary is created using curly braces `{}`.

Example:

```
student = {  
    'name': 'Christabel',  
    'age': 14,  
    'school': 'Silver Fountain College'  
}
```

General structure:

```
dictionary_name = {  
    key: value,  
    key: value,  
    key: value  
}
```

Each key and value are separated by a colon `:`.

Each key-value pair is separated by a comma `,`.

3. KEYS

A key is used to identify a value.

Example:

```
student = {  
  'name': 'Christabel',  
  'age': 14  
}
```

The keys are:

```
'name'  
'age'
```

Keys should be unique.

Example:

```
student = {  
  'name': 'Christabel',  
  'name': 'Belle'  
}
```

This is a bad idea because the second key replaces the first one.

4. VALUES

A value is the information stored under a key.

Example:

```
student = {  
  'name': 'Christabel',  
  'age': 14,
```

```
{  
    'coding': True  
}
```

The values are:

```
Christabel  
14  
True
```

Values can be almost any Python data type:

- Strings
- Integers
- Floats
- Booleans
- Lists
- Tuples
- Dictionaries
- Sets
- Even functions and objects

5. ACCESSING VALUES

We use square brackets with the key.

Example:

```
student = {  
    'name': 'Christabel',  
    'age': 14  
}  
  
print(student['name'])
```

Output:

```
Christabel
```

Another example:

```
print(student['age'])
```

Output:

```
14
```

Notice that we use the **key**, not a number.

Unlike lists:

```
names[0]
```

we use:

```
student['name']
```

6. ADDING NEW ITEMS

We can create a new key simply by assigning a value to it.

Example:

```
student = {  
    'name': 'Christabel'  
}  
  
student['hobby'] = 'Coding'
```

Now the dictionary becomes:

```
{  
    'name': 'Christabel',  
    'hobby': 'Coding'  
}
```

Python automatically creates the new key.

7. UPDATING VALUES

If the key already exists, assigning a new value changes the old value.

Example:

```
student = {  
    'age': 14  
}  
  
student['age'] = 15
```

Now:

```
print(student)
```

Output:

```
{'age': 15}
```

The old value has been replaced.

8. REMOVING ITEMS

There are several ways to remove information from a dictionary.

The method we learned first is:

```
.pop()
```

Example:

```
student = {  
    'name': 'Christabel',  
    'age': 14  
}
```

```
student.pop('age')
```

Now the dictionary becomes:

```
{  
    'name': 'Christabel'  
}
```

The key and its value are removed together.

9. WHAT DOES `.pop()` RETURN?

The `.pop()` method removes the key **and** returns its value.

Example:

```
student = {  
    'age': 14  
}  
  
removed = student.pop('age')  
  
print(removed)
```

Output:

```
14
```

Notice:

The dictionary loses:

```
'age': 14
```

But `.pop()` returns only:

```
14
```

This is one important difference between dictionaries and some other data structures.

10. CHECKING WHETHER A KEY EXISTS

We often need to know whether a key exists before using it.

Example:

```
student = {  
    'name': 'Christabel'  
}  
  
if 'name' in student:  
    print('Found!')
```

Output:

```
Found!
```

This prevents errors when trying to access keys that do not exist.

End of Part 1

In Part 2, we'll cover:

- Looping through dictionaries
 - `.keys()`
 - `.values()`
 - `.items()`
- Nested dictionaries
- The complete Student Management System explanation
- Best practices
- Summary and progress tracker